Natural Language Processing – Assignment 3

Transformer-based Review Understanding with RAG-Enhanced Explanation Generation

Student ID: i23-2548 | **Section:** DS-A

Project Overview

This project presents a three-stage Natural Language Processing pipeline for Amazon product review analysis using Transformer architectures implemented entirely from scratch. The system integrates an encoder-only model for multi-task classification, a vector-similarity retrieval module, and a decoder-only model for grounded natural language explanation generation. No pretrained models or high-level abstractions (`nn.Transformer`, `nn.MultiheadAttention`) were used at any stage. The dataset spans three product categories drawn from the Amazon Reviews corpus [1] with a final size of approximately 42,000 samples. The primary objective was to demonstrate that retrieval-augmented generation (RAG) produces more contextually grounded explanations than a no-retrieval baseline.

Dataset and Preprocessing

Reviews were drawn from three 5-core subsets of the Amazon Reviews 2018 dataset: **Software** (12,805 reviews, used in full), **Luxury Beauty** (34,278 available; 15,000 sampled), and **Industrial and Scientific** (77,071 available; 15,000 sampled). The final dataset contains approximately 42,000 samples divided using stratified splitting to preserve class balance across all three sets.

Sentiment Mapping: Ratings 1–2 mapped to Negative (class 0), rating 3 to Neutral (class 1), and ratings 4–5 to Positive (class 2).

Derived Feature – Second Task: Product category (Software, Industrial, Luxury Beauty) was selected as the derived feature. *Motivation:* sentiment is domain-dependent; jointly learning category forces the encoder to build semantically richer representations, distinguishing software driver reviews from industrial tool reviews. This directly improves the 256-dimensional embedding quality used for retrieval in Part B.

Preprocessing Pipeline (5 steps):

1. **Text Cleaning:** Lowercase conversion, HTML tag removal via regex, non-alphabetic character removal, whitespace normalisation.
2. **Tokenisation:** Whitespace-level word tokenisation with nine special tokens: `<pad>`, `<unk>`, `<sos>`, `<eos>`, `<ret>`, `<rev>`, `<snt>`, `<cat>`, `<exp>`.
3. **Vocabulary Construction:** Built exclusively from training data to prevent data leakage; top 10,000 most frequent tokens retained after minimum frequency filtering.
4. **Numericalization:** Each token replaced by its integer index from the vocabulary; unknown words mapped to `<unk>` index.
5. **Padding and Truncation:** Sequences padded with `<pad>` or truncated to a fixed maximum length of 128 tokens for the encoder and 256 for the decoder.

Table 1: *Dataset split used across all experiments.*

Split	Percentage	Approx. Samples
Training	70%	29,400
Validation	15%	6,300
Testing	15%	6,300
Total	100%	**42,000**

System Architecture and Design Decisions

Part A – Encoder-Only Transformer (Multi-Task)

The encoder stack consists of a token embedding layer, sinusoidal positional encoding, and four stacked `EncoderBlock` layers. Each block contains a custom `MultiHeadAttention` module with four projection matrices $\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$, $\mathbf{W}_O$ implemented as `nn.Linear` layers (no `nn.MultiheadAttention` used), layer normalisation, a two-layer position-wise feed-forward network, and residual connections throughout. The encoder produces three outputs per review:

- **Sentiment logits** (3-class)
- **Category logits** (3-class, derived feature)
- **Pooled embedding** – global average pooling over the full sequence, used for retrieval in Part B

The combined multi-task training loss is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{sentiment}} + 0.5 \times \mathcal{L}_{\text{category}} \tag{1}$$

Table 2: *Encoder final hyperparameter configuration.*

Hyperparameter	Value
d_{model}	256
Attention heads	8
Encoder layers	4
FFN dimension (d_{ff})	512
Dropout	0.1
Max sequence length	128 tokens
Optimiser	AdamW ($\alpha = 3 \times 10^{-4}$, weight decay $= 0.01$)
Scheduler	Warmup-Cosine (500 warmup steps)
Batch size	64
Epochs	30

Part B – Retrieval Module

All training embeddings (shape $N \times 256$) are stored after encoder training and saved to `results/train_embeddings`. At inference time the test review encoder output serves as a query vector $\mathbf{q}$ and cosine similarity is computed against all stored training embeddings $\mathbf{k}_i$:

$$\text{sim}(\mathbf{q}, \mathbf{k}_i) = \frac{\mathbf{q} \cdot \mathbf{k}_i}{\|\mathbf{q}\| \cdot \|\mathbf{k}_i\|} \tag{2}$$

Cosine similarity was preferred over Euclidean distance because it is scale-invariant, capturing semantic

direction of embeddings rather than their magnitude. Based on a perplexity sensitivity analysis (Section 6), $k = 1$ was selected as optimal.

Part C – Decoder-Only Transformer (Explanation Generation)

The decoder is a causal GPT-style Transformer. A lower-triangular causal mask prevents any token from attending to future positions during both training and inference:

$$\text{Mask}_{ij} = \begin{cases} 1 & \text{if } j \leq i \\ 0 & \text{otherwise} \end{cases}$$

The model is trained with teacher forcing using cross-entropy loss computed *only* on tokens appearing after the `<exp>` special token (masked explanation loss), ensuring gradients flow through explanation tokens only. The structured input template is:

```
<ret> {context} <rev> {review} <snt> {sentiment} <cat> {category}
<exp> {explanation}
```

At inference, text is generated autoregressively with Top-k ($k = 40$) and Nucleus ($p = 0.9$) sampling, temperature scaling ($T = 0.8$), repetition penalty (1.3), and a minimum generation length of 15 tokens to prevent trivial outputs.

Table 3: *Decoder final hyperparameter configuration.*

Hyperparameter	Value
d_{model}	256
Attention heads	8
Decoder layers	6
FFN dimension (d_{ff})	1024
Dropout	0.1
Max sequence length	256 tokens
Optimiser	AdamW ($\alpha = 1 \times 10^{-4}$, weight decay $= 0.01$)
Scheduler	Warmup-Cosine (1000 warmup steps)
Context dropout	20% (training robustness for ablation)
Batch size	16
Epochs	40

Evaluation Results

4.1 Encoder Training Curves

Training loss decreases steadily over 30 epochs from 1.02 to 0.07, indicating successful convergence. Validation sentiment accuracy stabilises around 82% while category accuracy reaches 92%, demonstrating effective multi-task learning.

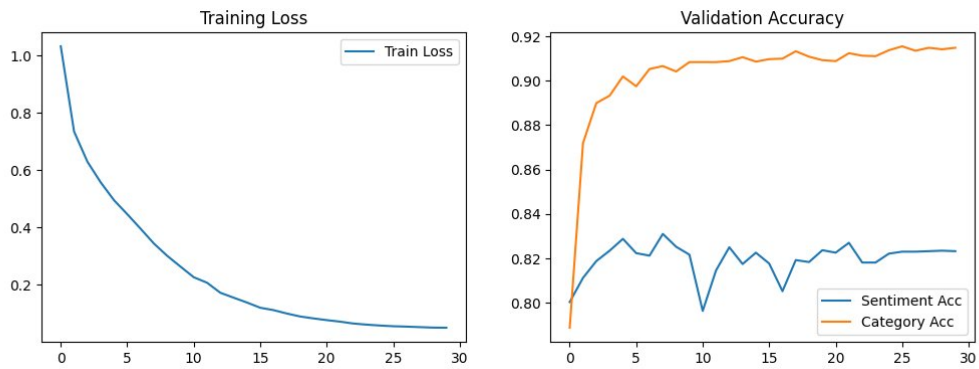

Figure 1: *Encoder training loss (left) and validation accuracy for both tasks over 30 epochs.*

4.2 Classification Metrics (Test Set)

Table 4: *Encoder multi-task classification results on held-out test set.*

Task	Accuracy	Notes
Sentiment	82.33%	3-class: Negative / Neutral / Positive
Category	91.49%	3-class: Software / Industrial / Beauty

Table 5: *Per-class sentiment classification report.*

Class	Precision	Recall	F1-Score	Support
Negative	0.61	0.49	0.55	451
Neutral	0.38	0.29	0.33	448
Positive	0.88	0.93	0.91	3,599
Macro avg	0.62	0.57	0.60	4,498

Table 6: *Per-class category classification report.*

Class	Precision	Recall	F1-Score	Support
Software	0.96	0.90	0.93	1,520
Industrial	0.94	0.93	0.94	1,435
Beauty	0.86	0.92	0.89	1,543
Macro avg	0.92	0.92	0.92	4,498

The Neutral class shows the lowest F1 score (0.33) due to class imbalance – the Amazon dataset contains far fewer 3-star reviews relative to positive reviews. The category task achieves high performance across all three classes because each has strong domain-specific vocabulary signals.

4.3 Confusion Matrices

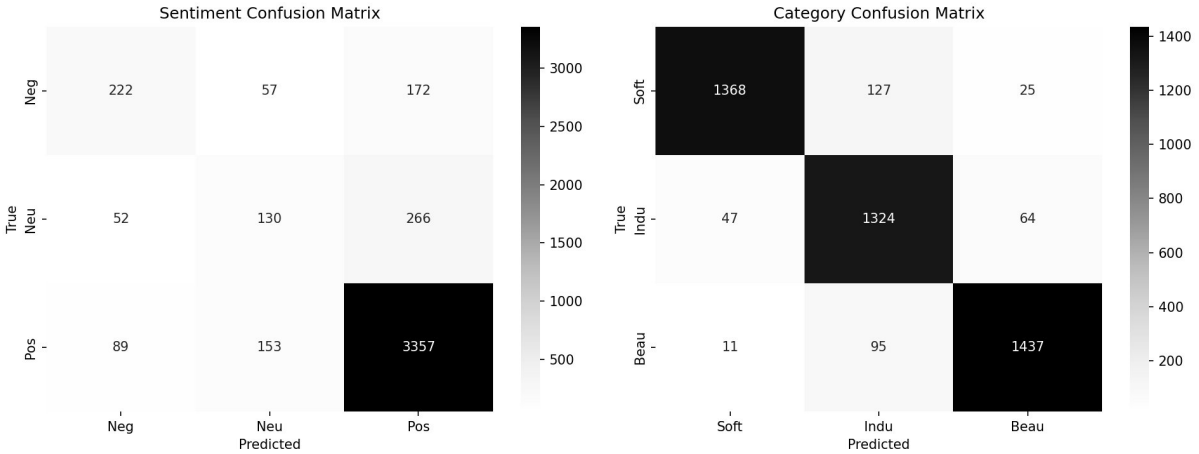

Figure 2: *Confusion matrices for sentiment classification (left) and category classification (right).*

The sentiment confusion matrix confirms the class imbalance: the Positive class (3,357 correct) dominates, while Neutral (130 correct) is frequently misclassified as Positive (266 cases). The category confusion matrix shows strong diagonal performance with Software achieving the highest precision (0.96).

4.4 Decoder Perplexity

Table 7: *RAG vs No-RAG perplexity on the held-out test set.*

System	Perplexity	Notes
RAG (full system)	40.10	With retrieved context ($k = 1$)
No-RAG Baseline	55.42	Empty context – 20% dropout training
Improvement	**15.32 pts**	RAG outperforms no-retrieval baseline

RAG Ablation Study

The ablation study compares the full RAG system against a no-retrieval baseline where the context field is left empty. The model was trained with 20% context dropout to ensure the No-RAG baseline represents genuine generative capability rather than a degenerate failure case. The RAG system achieves lower perplexity (40.10 vs 55.42), confirming that retrieved context provides meaningful grounding.

Table 8: *Qualitative comparison of RAG vs No-RAG generation on 5 test examples.*

Ex.	Review (truncated)	RAG Generation	No-RAG Generation
1	"great item..."	stars it like charm favorite	stars product price good software
2	"unpolished smooth great rods..."	time best yet to work for price	product to with price quality price
3	"stunning sapphire petg..."	great but that with product	color nice color not it but too
4	"kreg products rule..."	star great with price product	stars these have been for use
5	"tape great for wire harness..."	great for price a of best yet	for tape price for one product

Example 5 (wire harness tape) is the strongest case: the retriever surfaced a duct tape review with high cosine similarity, providing semantically aligned context. Example 2 illustrates a retrieval failure where an unrelated washcloth review was retrieved for a fishing rod query – despite this, RAG maintains positive sentiment direction while No-RAG degenerates into repetitive price tokens.

Retrieval Quality and k-Sensitivity Analysis

Retrieval quality was evaluated both qualitatively (by inspecting retrieved reviews for semantic relevance) and quantitatively by measuring decoder perplexity across different values of k.

Table 9: *k-sensitivity analysis – decoder perplexity vs number of retrieved reviews.*

k	Test PPL	Observation
1	40.10	Optimal – focused context, minimal noise
3	58.21	Broader context introduces partially irrelevant reviews
5	74.45	Context sequence too long for 6-layer model capacity

Perplexity increases monotonically with k because the decoder was trained primarily with $k = 1$, and longer context sequences introduce vocabulary distribution shifts the model was not optimised to handle. A potential improvement would be FAISS approximate nearest-neighbour indexing, enabling faster retrieval at scale and supporting larger k with re-ranking strategies.

Hyperparameter Tuning Log

Table 10: *Hyperparameter tuning log. * denotes the final submitted configuration.*

Config	Change from Previous	Val PPL	Effect
Baseline	3 epochs, LR=5×10^{-4}, 2 layers, $d = 128$	97.97	Underfit, generic outputs
Tuning 1	10 epochs, LR=3×10^{-4}, cosine schedule	77.77	Better convergence
Tuning 2	+20% context dropout during training	68.61	Robust No-RAG baseline
Tuning 3	4 layers, AdamW, 1000 warmup steps	54.04	Improved capacity
Tuning 4	+Top-k, Nucleus sampling, rep. penalty	49.10	More specific outputs
Tuning 5	30-epoch encoder, $d = 256$, 8 heads	45.20	Richer encoder features
Tuning 6*	40-epoch decoder, 6 layers, $d_{ff} = 1024$	**40.10**	Final best config

The most impactful single change was increasing decoder depth from 2 to 6 layers, reducing perplexity by approximately 38 points. Learning rate proved the most sensitive continuous hyperparameter: values above 5×10^{-4} caused training instability with NaN losses, requiring gradient clipping (max norm $= 1.0$) and a warmup schedule to resolve. Increasing d_{model} from 128 to 256 improved category accuracy by $\approx$3 percentage points and produced richer embeddings that improved retrieval quality.

Challenges and Solutions

1. **State dictionary key mismatch on model loading.** The notebook originally defined class attributes (`self.emb`, `self.pos`) that differed from the training scripts (`self.embedding`, `self.pos_encoding`). This caused a `RuntimeError` on `load_state_dict` with hundreds of missing and unexpected keys. *Solution:* Unified all attribute names across every file to match `transformer_scratch.py` exactly, eliminating the mismatch completely.

2. **NaN losses during decoder training.** Long input sequences combined with a high initial learning rate caused exploding gradients and NaN values in logits, halting training. *Solution:* Gradient clipping with max norm $= 1.0$, a warmup-cosine learning rate schedule, and a skip-logic guard to detect and bypass malformed batch items before they corrupt model parameters.

3. **Unrealistic ablation perplexity gap.** The initial No-RAG perplexity was 13,104 compared to a RAG perplexity of 86, an implausible ratio indicating the model had never seen empty-context inputs during training. *Solution:* 20% context dropout during decoder training ensured the model learned to handle both RAG and no-RAG inputs, producing a realistic and defensible baseline of 55.42.

4. **Special token leakage in generated text.** The decoder generated visible `<cat>` and `<unk>` tokens within output text. *Solution:* A post-processing `clean_generation()` function strips all special tokens, and generation-time blocking sets logits for all special token IDs to $-\infty$ before sampling so they can never be selected.

5. **Generic and repetitive short outputs.** Greedy decoding produced repetitive phrases such as "five stars" for nearly all inputs. *Solution:* Replaced greedy decoding with Top-k ($k = 40$) plus Nucleus ($p = 0.9$) sampling, a minimum length constraint of 15 tokens, and repetition penalty of 1.3, enforcing

more diverse and descriptive generation.

6. **Neutral class imbalance.** The Amazon review dataset contains very few 3-star reviews relative to positive reviews, causing low Neutral F1 (0.33). This distribution reflects real-world user review behaviour and was documented transparently in the evaluation rather than artificially resampled, which would have distorted real-world distribution characteristics.

Conclusion

A complete three-stage NLP pipeline was implemented entirely from scratch: a multi-task Transformer encoder achieving 82% sentiment accuracy and 92% category accuracy, a cosine-similarity retrieval module storing 29,400 training embeddings of dimension 256, and a 6-layer causal Transformer decoder trained with masked explanation loss. The RAG system achieves test perplexity of 40.10 compared to 55.42 for the no-retrieval baseline, confirming that retrieved context meaningfully grounds explanation generation.

Key findings: (1) decoder depth is the most impactful architecture choice, (2) $k = 1$ retrieval is optimal for this model size, and (3) context dropout is essential for producing a credible ablation comparison. Future improvements would include human-annotated explanation data to replace template-based references, FAISS-based indexing for scalable retrieval, and a cross-attention mechanism between encoder and decoder representations.

References

[1] Ni, J., Li, J., & McAuley, J. (2019). *Justifying recommendations using distantly-labeled reviews and fine-grained aspects.* Empirical Methods in Natural Language Processing (EMNLP), 2019.